

CSCE 627 THEORY OF COMPUTABILITY · MODULE 10 · LECTURE NOTES

Logic and Incompleteness

Gödel, as a corollary.

MODULE OUTCOMES

Explain what a formal system is and what provability means.

Derive incompleteness from undecidability.

State both incompleteness theorems precisely.

Explain what they do and do not establish.

Explain the consequences for verification.

1 Formal systems

A FORMAL SYSTEM MAKES PROVABILITY MECHANICAL, WHICH IS THE POINT

Axioms, inference rules, and proofs that are finite sequences of statements, each of which follows from earlier ones by a rule. Nothing in a proof requires insight to *check*.

So *checking* a proof is decidable — it is a purely syntactic verification — **and that is exactly what makes formalisation worth the considerable trouble it takes.** A proof assistant is a proof checker.

Which means the set of provable statements is recognisable (Module 04 §2): enumerate all finite strings in order, check each for being a valid proof, and every theorem eventually appears. **Slow, and a genuine enumeration.**

Recognisable, and the question is whether it is also *decidable* — can you determine that a statement is *not* provable, in finite time? **That is incompleteness restated in computability terms**, and putting it that way is what makes §2's derivation available.

2 Incompleteness from undecidability

```

SUPPOSE a formal system F is
  SOUND      -- it proves only true statements
  COMPLETE  -- it proves every true statement
and strong enough to describe Turing machines

"M does not halt on w" is a statement about arithmetic
(Gödel's arithmetisation, or Module 09's encodings, make
this precise).

Then a halting decider would exist:
  given (M, w), enumerate ALL proofs in F
  if a proof of "M halts on w" appears, answer YES
  if a proof of "M does not halt on w" appears, answer NO

By COMPLETENESS one of the two statements is true and
therefore provable, so the search terminates.
By SOUNDNESS whichever proof appears is correct.
So this procedure DECIDES HALTING.

But Module 05 proved that no such procedure exists.

THEREFORE: no sound, complete, sufficiently strong formal
system exists. Any sound system strong enough to talk about
computation is INCOMPLETE -- there are true statements it
cannot prove.

THAT IS GÖDEL'S FIRST INCOMPLETENESS THEOREM, derived from
halting in half a page.

```

Incompleteness is a corollary of undecidability — which is the cleanest available route to it and is why this module comes after Module 05 rather than before. It is also the intellectual high point of the course: a result usually presented as profound and mysterious becomes a short argument from a theorem you proved four modules ago.

AND GÖDEL'S ORIGINAL ROUTE, FOR CONTRAST

Gödel, in 1931 and five years before Turing, constructed a sentence asserting its own unprovability — informally, 'this statement is not provable in F'.

If F proves it, then F has proved a false statement, so F is unsound. If F proves its negation, F has proved that the statement is provable, which is also false. So F proves neither, and the statement is true and unprovable in F.

The hard part was *arithmetisation* — encoding statements, proofs, and the relation 'is a proof of' as arithmetic predicates about numbers, so that the system could talk about its own syntax. Which is Module 04 §3's programs-as-data encoding, invented independently and five years earlier, for a different purpose.

So both routes are Module 05 §3's *diagonalisation* — construct the object that differs from everything the enumeration covers — and Gödel essentially invented the techniques of computability theory in order to prove a theorem about logic. Which is why the two subjects are inseparable, and why Turing's paper five years later reads as a continuation.

3 Both theorems, stated carefully

	First incompleteness theorem	Second incompleteness theorem
What it says	Any consistent, sufficiently strong, effectively axiomatised formal system is incomplete — there are true statements of its language it cannot prove.	Any such system cannot prove its own consistency.
What it requires	Enough arithmetic to encode computation, and an effectively enumerable axiom set — both conditions are essential.	The same conditions, plus the ability to formalise provability inside the system.
Escape route 1	Weaker systems are complete — Presburger arithmetic (addition without multiplication) is complete <i>and</i> decidable, and so is the first-order theory of the reals.	A stronger system can prove a weaker one consistent — set theory proves arithmetic consistent, and the regress simply continues upward.
Escape route 2	A non-effective axiom set is complete — take all true arithmetic statements as axioms. And useless, since you cannot recognise an axiom.	Consistency can be assumed rather than proved, and in practice always is — which is what a trusted base amounts to (§4).

The escape conditions matter and are exactly what popular accounts omit. Incompleteness is not a statement about all of mathematics; it is a statement about systems strong enough to encode computation and weak enough to be mechanically checkable — and that pair of conditions is what the theorem is really about.

WHAT THE THEOREMS DO NOT SAY

They do not say there are truths nobody can ever know. A statement unprovable in F may be perfectly provable in a stronger system, and routinely is — Goodstein's theorem is unprovable in Peano arithmetic and provable in set theory, which is an ordinary mathematical situation rather than a limit on knowledge.

They do not say mathematics is broken, arbitrary, or unreliable. Essentially all working mathematics is provable in standard set theory, and the independent statements are mostly carefully constructed or drawn from set theory itself.

They do not apply to systems too weak to encode computation. Presburger arithmetic is complete and decidable — which is precisely why SMT solvers can handle linear integer arithmetic and cannot handle general arithmetic (CSCE 625 Module 08 §4's decidable fragments), so the escape route is a load-bearing engineering fact rather than a technicality.

And they say nothing whatsoever about human minds. The arguments that they establish something about human intelligence require assuming that humans are both consistent and formalisable — which is precisely what would have needed establishing, so the argument assumes its conclusion. This is worth being able to state, because the claim is common.

4 Consequences for engineering

- **No verification system can prove every true property** of the programs in its scope — **so a verifier's failure to prove something does not mean it is false**, and treating a failed proof as a bug report is a category error.
- **Which is why proof assistants are interactive rather than automatic. The human supplies the search that no complete automatic procedure could perform** — which is **Module 08 §2's sound-and-complete-but-not-terminating row**, and it is the only position that gets both soundness and completeness.
- **So decidable logical fragments are extremely valuable.** Presburger arithmetic, linear real arithmetic, bit-vectors, arrays, uninterpreted functions, the effectively-propositional fragment. **This is what SMT solvers are assembled from**, and the engineering is in staying inside the decidable parts.
- **And a verified system's guarantee is relative to its axioms**, which include a formal model of the hardware, the compiler, and the specification — any of which may be wrong. **A proof about a model is not a proof about a machine.**
- **So 'proved correct' means 'proved correct relative to this specification, in this logic, assuming this model of the machine'.** That is the **practical consequence**: every verification result has a trusted base, **and the interesting question is always what is in it** — which is this program's state-your-assumptions discipline in the one subject where the assumptions are axioms.

WHY THIS MODULE IS HERE

Incompleteness is usually taught as a result about logic and presented as mysterious. Derived from halting it is neither — it is a corollary, and §2's proof fits on a slide.

And the historical direction of discovery was the reverse: Gödel's arithmetisation came first, and Turing formalised computation partly to settle a question that Gödel's work had raised (Hilbert's decision problem for first-order logic). **The corollary is older than the theorem it follows from.**

So the two subjects are one subject. Provability is a computational notion — it is recognisability (§1) — and computation is a logical one, and the limits of each are the limits of the other.

Which is why Module 09 §4's BB(748) result is the right preparation: a question about one specific finite integer that standard set theory cannot settle is incompleteness made as concrete as it is possible to make it, and it removes any suspicion that the phenomenon concerns only artificial self-referential sentences.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sipser — section 6.2	§2's derivation of incompleteness from undecidability , which is the route this module takes.
Smith — An Introduction to Gödel's Theorems (free study guide)	The careful treatment of §3 , with the conditions stated properly. The author's free Teach Yourself Logic guide is also excellent.
Nagel & Newman — Gödel's Proof	The short classic account of §2's original route , including arithmetisation. Library copy, and under 150 pages.
Franzén — Gödel's Theorem: An Incomplete Guide to Its Use and Abuse	§3's callout, at book length — the standard correction of the misreadings, and worth reading before citing the theorems anywhere.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write out the derivation of incompleteness from halting** in your own words.
- 2 **Identify exactly which steps use soundness and which use completeness.**
- 3 **Explain why the provable statements are recognisable** and what it would take for them to be decidable.
- 4 **State both theorems precisely**, including every condition.
- 5 **For each condition, give a system that fails it** and is therefore complete.
- 6 **Use an SMT solver on Presburger arithmetic** and then on a formula requiring multiplication. Report what happens.
- 7 **Find a popular claim about Gödel's theorems** and assess it against §3's callout.
- 8 **Take a verified system** (seL4, CompCert) and find its stated trusted base.
- 9 **List what is in that base** and what could be wrong in each item.
- 10 **Write the honest form of 'proved correct'** for a verification you would want to perform.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What makes proof-checking decidable, and what does that imply about provability?
- 2 Derive incompleteness from undecidability.
- 3 Where do soundness and completeness each enter the argument?
- 4 Describe Gödel's original route and name the hard part.
- 5 State both theorems with their conditions.

- 6 Give two escape routes for each.
- 7 Name four things the theorems do not say.
- 8 Why can SMT solvers handle linear arithmetic?
- 9 What does 'proved correct' actually mean?